Gas Combustion Model Instructions

James Karr

July 14, 2026

1 Converting a Cantera YAML mechanism to XML

IMPORTANT: All `.yaml` mechanism files must be converted to `.xml` using the `yaml2xml.py` script in order to match the expected formatting the `gasCombustion` model expects!

The `gasCombustion` model does not read Cantera YAML mechanism files directly – it reads the CTML-like XML format produced by the `yaml2xml.py` script in this directory. Any mechanism exported from Cantera (e.g. `cantera.Solution.write_yaml()`, or one of the mechanisms distributed with Cantera) must be converted before it can be referenced from a `.ups` file.

Note: the standard mechanisms shipped with Cantera that fit the criteria for this model (gas-phase combustion – e.g. GRI-Mech, H2/O2) have already been converted and are available in this directory. Check here before converting your own copy of a standard mechanism.

1.1 Python dependencies

`yaml2xml.py` and the `transportFit.py` helper it calls need the `numpy`, `ruamel.yaml`, and `yaml` (PyYAML) Python packages in addition to other standard packages typically shipped with a standard python download. If they are not already installed, get them with:

```
pip install numpy ruamel.yaml pyyaml
```

1.2 Usage

Run the script on one or more YAML mechanism files:

```
python3 yaml2xml.py mech1.yaml [mech2.yaml ...] [-o OUTPUT_DIR]
```

For each input file, an XML file is written next to it (or into `OUTPUT_DIR` if `-o` is given) with the same base name and a `.xml` extension. The `<mechanism>` tag's `name` attribute is set to the input file name with the `.yaml` extension stripped.

For example, converting the Burke 2012 hydrogen mechanism used by the sample input files in this directory:

```
python3 yaml2xml.py Burke2012Modified.yaml
# wrote Burke2012Modified.xml
```

This can be done on any machine and only needs to be done once per mechanism file. The `.xml` mechanism file created is the only file needed with Uintah to run the gasCombustion model.

For example, one might run the `yaml2xml.py` converter script on their local computer with python and all dependencies installed then copy the outputted `.xml` mechanism to their cluster with Uintah installed, avoiding the need to install one-time-use python packages on the cluster.

1.3 What the script does

- Translates each Cantera `phase`, `species` (composition, NASA7 thermo polynomials, Lennard-Jones transport parameters), and `reaction` (elementary, three-body, falloff with Troe/SRI/Lindemann forms, and PLOG) entry into the equivalent XML element.
- Carries over inline `# ...` comments from the YAML (such as the `# Reaction N` tags Cantera writes next to each reaction) as XML comments on the corresponding element.
- Calls `transportFit.py` to fit five-coefficient polynomial curves for viscosity, thermal conductivity, and binary diffusion coefficients from each species' kinetic-theory transport parameters, and embeds them as `<viscosityFit>`, `<conductivityFit>`, and `<binaryDiffusionFits>` elements. **Every species in the YAML file must have a `transport` block** – if the mechanism was exported without transport data, `gasCombustion` will refuse to parse the resulting XML.

Any reaction with a rate expression the script does not recognize is not silently dropped: the script prints a warning to stderr and copies the raw YAML data into a `<raw>` element instead of an `<Arrhenius>` block, so the omission is visible when the XML is inspected.

Once the `.xml` file is written, point the `gasCombustion` model's `<mechanismFile>` tag at it (see Section 2).

2 The `gasCombustion` input block

The model is enabled in a `.ups` file by adding a `<Model type="gasCombustion">` entry inside `<Models>`, containing a `<gasCombustion>` block. A minimal example:

```
<Models>
  <Model type="gasCombustion">
    <gasCombustion>
      <material>source</material>
      <mechanismFile> inputs/ICE/RX_MODELS/Burke2012Modified.xml </mechanismFile>
      <closureSpecies> N2 </closureSpecies>
      <!-- Stoichiometric hydrogen-air, tracked-species order -->
      <Y_init> [0.0, 0.0285, 0.0, 0.0, 0.0, 0.2264, 0.0, 0.0] </Y_init>
    </gasCombustion>
  </Model>
</Models>
```

2.1 Required tags

- `material` – name of the ICE material this model acts on (matches the name given in the material's `<geom_object>` block elsewhere in the `.ups` file).
- `mechanismFile` – path to the XML reaction mechanism produced by `yaml2xml.py` (Section 1). All species and reaction data come from this file; the `.ups` file only supplies solver

controls and initial conditions. Total path is needed if not rooted from the StandAlone directory.

- `closureSpecies` – the one species from the mechanism whose mass fraction is set by closure ($Y = 1 - \sum$ of the others) instead of being transported directly. Normally the inert/bath species (e.g. `N2`, `Ar`); if the mixture has none, pick a dominant product or reactant instead.
- `Y_init` – background initial mass fractions for every *tracked* species (all mechanism species except the closure species), in mechanism order. The list length must equal the number of tracked species, and the values must not sum to more than 1.

2.2 Optional solver-control tags

Tag	Default	Meaning
`doChemistry`	`true`	Enable/disable the chemistry substep.
`doDiffusion`	`true`	Enable/disable diffusion (thermal, momentum, molecular).
`debug`	`false`	Extra diagnostic output.
`rtol`	`1e-12`	Chemistry integrator relative tolerance.
`atol_Y`	`1e-12`	Absolute tolerance, species mass fractions.
`atol_T`	`1e-12`	Absolute tolerance, temperature.
`safety`	`0.9`	Step-size safety factor.
`max_grow`	`2.0`	Max factor to grow the chemistry sub-step.
`max_shrink`	`0.1`	Max factor to shrink the chemistry sub-step.

2.3 Optional initialization blocks

`<geom_object>` One or more geometry-piece blocks (using the standard Uintah `GeometryPieceFactory` syntax) may be nested inside `<gasCombustion>` to set non-background initial mass fractions in a region of the domain:

```
<geom_object>
  <box label="ignitionRegion">
    <min> [0,0,0] </min>
    <max> [0.001, 0.01, 0.01] </max>
  </box>
  <Y> [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0] </Y>
</geom_object>
```

As with `Y_init`, `<Y>` must have one entry per tracked species (mechanism order, excluding the closure species) and must not sum to more than 1.

`<initProfile>` Initializes temperature, velocity, density, pressure, and species mass fractions from a 1-D profile file (e.g. a laminar flame profile), overriding any `geom_object` blocks when present:

```
<initProfile>
  <filename> /path/to/flameProfile.dat </filename>
  <axis> x </axis>
</initProfile>
```

`filename` is a whitespace-delimited text file where each row is `x T u rho p Y_1 Y_2 ... Y_N` (one column per tracked species, in mechanism order); lines starting with `#` are treated as comments. `axis` selects which coordinate direction (`x`, `y`, or `z`) the profile is applied along.